

Spectral Clustering from scratch: deflation and inverse power method for clustering data

Massimiliano Carli
Politecnico di Torino
Torino, Italy
s337728@studenti.polito.it

Rosario Interlandi
Politecnico di Torino
Torino, Italy
s334202@studenti.polito.it

Abstract—In this report, we present a comprehensive method for spectral clustering, covering the entire process from the computation of the required matrices and their respective eigenvalues (implementing deflation and inverse power method, also providing pseudocode for the former), to the final clustering results obtained through various adjustments specifically fine-tuned on three sample synthetic datasets. Our goal is to provide a detailed guide to the spectral clustering procedure, highlighting key steps in the procedure.

Index Terms—spectral clustering, inverse power method, deflation method, eigenvalues computation

I. INTRODUCTION

Clustering is a fundamental task in data analysis and unsupervised machine learning, the main goal is to cluster data points into distinct groups based on some metric that describes their similarities, which can be different for each specific application.

Traditional clustering methods, such as *k-means*, rely on some assumptions about cluster shapes, this approach can limit their capability to more complex datasets. Spectral clustering offers a more flexible approach by developing a graph-based representation of data, in which edges of the graphs store the information about similarity of data points. This graph is then represented by a matrix, whose eigenvalues and eigenvectors provide meaningful information that can be exploited to achieve the desired goal.

In this report, we present an implementation of spectral clustering from scratch, describing the construction of required matrices, the computation of its eigenvalues and eigenvectors using the *deflation method* and *inverse power method*, and the final clusterings obtained as a result of our work.

We validated our approach using three synthetic datasets: (i) *Spiral*, (ii) *Circles* and (iii) *Torus*, representing shapes that make traditional clustering methods perform poorly. Additionally, we optimized hyperparameters of models employed during the clustering pipeline to achieve the best performance for our method.

II. LAPLACIAN MATRIX CONSTRUCTION

A. Similarity matrix

Given a dataset of n points described by m features, we initially constructed the similarity matrix $S \in \mathbb{R}^{n \times n}$, where

each element s_{ij} represents the similarity between two points. s_{ij} is given by the *Gaussian similarity function*

$$s_{ij} = \exp\left(-\frac{\|x_i - x_j\|^2}{2\sigma^2}\right) \in (0, 1]$$

where we assumed a unitary standard deviation, (i.e. $\sigma = 1$), and the use of 2-norm. Different values of σ or different definitions of the norm could be used based on specific use cases.¹

This implies that pairs of points with a square Euclidean distance less than one will show a high similarity - with the edge case of $i = j$ for which $\|x_i - x_j\|^2 = 0$, and therefore $s_{ij} = 1$, while the similarity will approach zero for larger Euclidean distances. Furthermore, this similarity implementation provides the property that $s_{ij} = s_{ji}$ hold for any i and j , making S symmetric.

As mentioned before, this specific implementation of similarity is arbitrary, different formulas could be used based on specific use cases and necessities.

B. Knn algorithm for connections reduction

If we consider each point of our dataset as a node in a graph, the matrix S would represent the weighted adjacency matrix of a non-oriented completely connected graph, despite many of these connections being weak, i.e. arbitrary near to zero.

To eliminate such noise, we built a new matrix W , given by S , in which we preserved only the most meaningful connections. Zeroing out all non-meaningful entries of S was achieved by applying the *KNN method* with an arbitrary k . More specifically, if at least one of two given nodes i and j contains the other in its KNN, then the connection between the two is preserved; otherwise, it is deleted. Additionally, W 's diagonal (similarity of each point with itself, which is always equal to 1) was set to zero. It is worth noting that this approach preserves the symmetry of the matrix, making W also symmetric.²

¹Note: The implementation of the Gaussian similarity can be simplified by expressing the squared Euclidean distance in terms of basic algebraic matrix operations.

²Note: Since W being probably sparse, it is convenient to implemented sparsity storage for increasing convergence and saving computer memory.

C. Degree and Laplacian matrices

After calculating W , we constructed the diagonal matrix D by

$$d_{i,i} = \sum_{j=1}^n W(i,j)$$

The final Laplacian matrix is then defined as $L = D - W$, where both D and W are symmetric. As a result also L will be symmetric.

To understand the structure of the Laplacian matrix, we can analyze its pattern, examples of which are shown in Figure 3 for one of datasets discussed later. Typically, the Laplacian matrix exhibits a sparse pattern, with the most densely populated regions appearing as multiple different blocks or well defined shapes disposed along the diagonal, and/or the upper and lower triangular regions of the matrix. These dense regions correspond to areas in the graph where nodes are highly connected, i.e. high similarity between points.

Additionally, there are far smaller, but still dense regions found in the upper and lower triangular parts of the matrix. These regions suggest a group of nodes that act as connectors between two distinct dense regions on the diagonal.

D. Normalized laplacian matrix

To maintain the symmetry of W , some nodes have more than k connections. To prevent nodes with more connections from contributing disproportionately to the spectral properties, we updated our Laplacian matrix with its normalized form

$$L_{norm} = D^{-1/2} L D^{-1/2} = I_n - D^{-1/2} W D^{-1/2}$$

Here, for each pair of nodes (i, j) , the contribution of their connection is given by

$$\frac{w_{ij}}{\sqrt{d_{ii}} \sqrt{d_{jj}}}$$

This normalization ensures that the contribution of each connection is independent of the number of connections each node has.

III. COMPUTATION OF THE M SMALLEST EIGENVALUES AND EIGENVECTORS

To perform spectral clustering, we need the M smallest eigenvalues and eigenvectors associated to the Laplacian matrix L , with M and arbitrary integer.

To compute the eigenvalues and eigenvectors of L , we applied the inverse power method, together with the deflation method. The former is an iterative method that, given an initial guess for an eigenvalue-eigenvector pair of a matrix, progressively improves the approximation of both; the latter also an iterative method that, given a matrix and a procedure to extract just one eigenvalue and eigenvector pair, is able to recursively extract all remaining eigenvalue and eigenvector pairs of the given matrix.

A. Inverse power method

We initialize the initial guess for each eigenvalue-eigenvector pair by setting the eigenvalue to zero and the eigenvector to a vector of all ones. Then, at each iteration, we solve the system

$$(L - pI) v_{k+1} = v_k,$$

where p denotes the estimated smallest eigenvalue (initially set to 0). As the iterations progress, the vector v_k converges to the eigenvector corresponding to the smallest eigenvalue of L^{-1} . The process is repeated until either the maximum number of iterations is reached or the relative difference between consecutive eigenvalues falls below a specified tolerance.

B. Deflation Method

Once the first eigenvalue and eigenvector pair is extracted, the deflation method proceeds by constructing a Householder reflector that zeroes out all but one component of the column of A under iteration i , leaving on the i -th entry of the column an eigenvalue. The reflector is defined as

$$P = I - 2 \frac{uu^T}{u^T u}$$

where

$$u = x_i + e_i$$

and x_i is the eigenvector provided by Inverse Power method at iteration i .

This reflector is then used to transform the matrix (L) into a reduced form, isolating the next eigenvalue and eigenvector pair in a smaller sub-matrix. At each step, the current eigenvalue is stored, and the eigenvector is corrected to account for its dependence on previously computed eigenvectors.

If the initial guess for eigenvalue in the inverse power method is properly set (zero for the first iteration, and the eigenvalue just computed for all subsequent iterations ³), the deflation method will return eigenvalues (and the corresponding eigenvectors) in ascending order. A pseudocode of the described method is provided in Algorithm 1.

C. Eigenspace

We will select only the smallest M eigenvalues, using the corresponding eigenvectors as columns of a new matrix U , which represent the transformed dataset in the reduced eigenspace.

This will allow us to map the data to a reduced-dimensional eigenspace, where points with significantly higher similarity will form geometrically distinct clusters. Such behavior of the eigenspace representing our desired shape of the distribution of data can be observed in Fig. 1, where the same exact dataset is represented in both its canonical representation, and the corresponding M -reduced eigenspace representation.

³choosing constantly zero at every iteration would still work, updating the guess to the just computed eigenvalue only helps convergence of inverse power method, while exploiting the property of deflation for which it computes eigenvalues in ascending orders

Algorithm 1 Pseudocode for Deflation using Inverse Power Method for computing the eigenvalues of a matrix A . The method iteratively extract each eigenvalue and eigenvector pair from A , reducing A until the required number of eigenvalues and eigenvectors pairs are obtained. Since we assume A symmetric, we can use the initial guess of eigenvector $p = 0$ to extract eigenvalues in ascending order, line 17 is provided to speed up convergence for the execution of inverse power method at next iteration, since the eigenvalue at following iteration $i + 1$ can not be smaller than the one just extracted at iteration i . **Note:** $1_m = [1, 1, \dots, 1] \in \mathbb{R}^m$; $0^{m \times l}$ is a matrix of only zeros, with size $m \times l$; MATLAB-like notation has been used.

Require: Symmetric matrix $A \in \mathbb{R}^{n \times n}$, Number of eigenvalues $M \in \mathbb{N}$ s.t. $0 < M \leq n$

```

1:  $D \leftarrow 0^{M \times M}$ ,  $V \leftarrow 0^{n \times M}$ .
2:  $B \leftarrow A$ 
3:  $P_{total} \leftarrow I_n$ 
4:  $p \leftarrow 0$ 
5:  $v_0 \leftarrow 1_n$ 
6:  $i \leftarrow 1$ .
7: while  $i \leq M$  do
8:    $A_i \leftarrow B(i : \text{end}, i : \text{end})$ .
9:   Compute eigenvector  $\bar{x}_i$  using Inverse Power Method on  $A_i$  with  $p$  and  $v_0$  guesses
10:   $u \leftarrow \bar{x}_i + e_i$ 
11:   $\bar{P} \leftarrow I - 2 \frac{uu^T}{u^T u}$  {Householder reflector}
12:   $P \leftarrow \text{block diagonal}(I_{i-1}, \bar{P})$ . {Preserve previously computed eigenvectors}
13:   $B_{\text{prev}} \leftarrow B$ 
14:   $B \leftarrow PBP$  {Apply reflector}
15:   $\lambda \leftarrow B[i, i]$ .
16:   $D[i, i] \leftarrow \lambda$ .
17:   $p \leftarrow \lambda$  {Speeding up convergence}
18:   $x_{\text{correction}} \leftarrow 0 \in \mathbb{R}^{i-1}$  {Build eigenvector correction}
19:  for  $j = i - 1$  to  $1$  step  $-1$  do
20:     $\lambda_{\text{prev}} \leftarrow D(j, j)$ 
21:    if  $\lambda \neq \lambda_{\text{prev}}$  then
22:       $b_{\text{corr}} \leftarrow B_{\text{prev}}(j, j + 1 : \text{end})$ 
23:       $x_{\text{correction}}(j) \leftarrow -\frac{b_{\text{corr}} \cdot [x_{\text{correction}}(j+1 : \text{end}); x_{\text{bar}}]}{\lambda_{\text{prev}} - \lambda}$ 
24:    else
25:       $x_{\text{correction}}(j) \leftarrow 0$  {Avoid division by zero}
26:    end if
27:  end for
28:   $x_{\text{corrected}} \leftarrow [x_{\text{correction}}; x_{\text{bar}}]$ 
29:   $x \leftarrow P_{total} \cdot x_{\text{corrected}}$  {Transform back to original space}
30:   $V(:, i) \leftarrow x$ 
31:   $P_{total} \leftarrow PP_{total}$ 
32:   $i \leftarrow i + 1$ 
33: end while
34: return  $D, V$ 

```

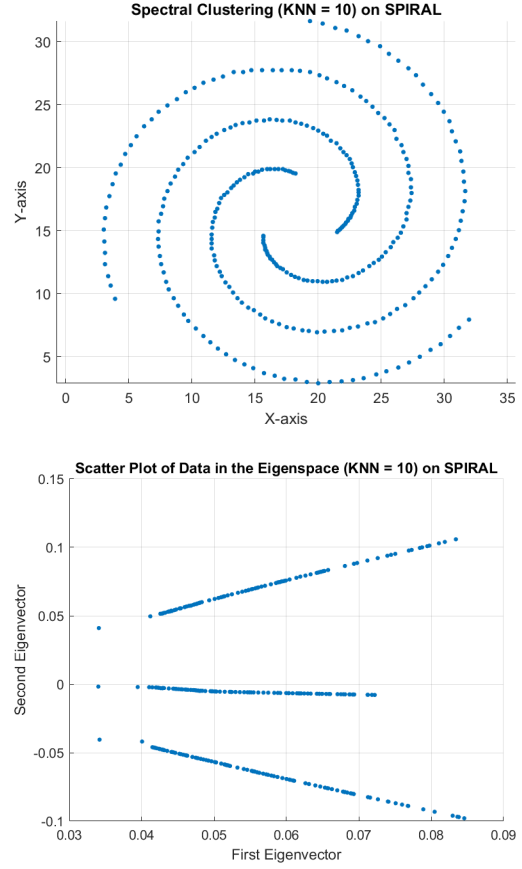


Fig. 1. Spectral clustering results on the spiral dataset using KNN = 10. The top plot shows the original data distribution, while the bottom plot visualizes the data in the eigenspace, highlighting the separation into clusters.

The final step will be applying a traditional clustering algorithm to this exact reduced eigenspace-transformed dataset, and return the resulting indexing of points in their respective groups as our final result.

IV. RESULTS

To test our method, we used three synthetic datasets, which data distribution can be seen in Figures 4, 5, and 6, together with the results of our clustering:

- **Dataset (i): Spiral** - Represents three sets of points arranged on a 2-dimensional plane to form a spiral.
- **Dataset (ii): Circle** - Represents two concentric circular figures, along with a cloud of points located nearby.
- **Dataset (iii): Torus** - Represents two three-dimensional concentric toroidal sets of points.

A comparison with the same clustering results obtained with a direct k-means clustering on the original data can be appreciated in Figure 7.

A. Fine-tuning

1) *Tuning of M and KNN:* We tested different values of k for the KNN method to reduce the noise on the original similarity matrix S , namely $k = \{10, 20, 40\}$. The algorithm performed well on all three datasets when a reasonable number

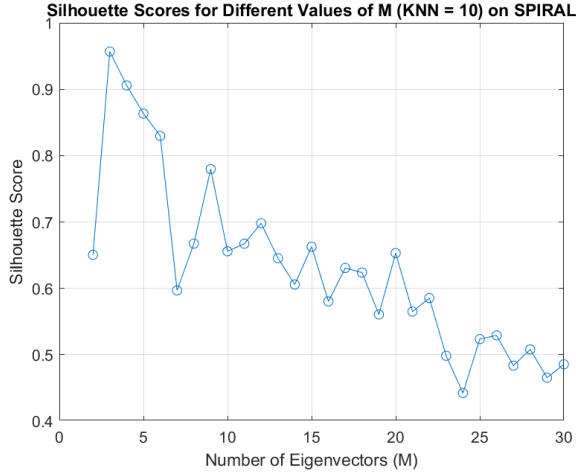


Fig. 2. Silhouette scores for spectral clustering on the spiral dataset with KNN = 10, plotted against the number of eigenvectors (M). The graph shows how clustering performance varies with different numbers of eigenvectors used for embedding.

of clusters M was set (according to the intuitive distribution we can make just by looking at the scatter plot, i.e. $M = 3$ for datasets (i) and (ii), and $M = 2$ for the dataset (iii)).

However, performance worsens progressively as M increased. Such behavior can be seen in Figure 2, where different silhouette scores are compared with a progressively increasing value of M . This is due to the fact that increasing M forces the algorithm to partition the data into more clusters that may be split arbitrarily, this forced fragmentation lowers the silhouette score and the overall quality of the clustering, as it no longer reflects the inherent structure of the data.

Furthermore, we observed that for an increasing value of k the Laplacian sparsity pattern got more complex, increasing the density of the same regions which presented non-zero elements. We justify this behavior since a higher value of KNN includes more neighbors in the construction of the similarity graph, resulting in a denser graph structure, and a denser Laplacian matrix as a consequence. The comparison between the Laplacian sparsity pattern of Spiral with different values of k can be appreciated in Figure 3.

However, this also introduces more noise, potentially reducing the clarity of the clustering boundaries if more complex datasets were presented. As a result, we adopted $k = 10$ and $M = \{2, 3\}$ for the remaining analysis.

2) *Tuning of clustering method on the eigenspace:* In addition to tuning k for KNN and M for the number of smallest eigenvalues, we also experimented with different traditional clustering methods applied to the reduced eigenspace dataset. More specifically, we implemented:

- K-means
- Hierarchical clustering

While K-means required no tuning, since the only parameter required by it (k) was always set equal to M ; Hierarchical clustering was fine-tuned automatically using an algorithm that tested all possible linkage method among $\{\text{ward, single,}$

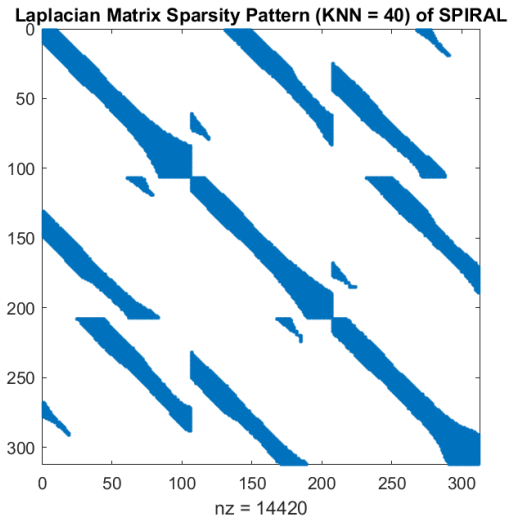
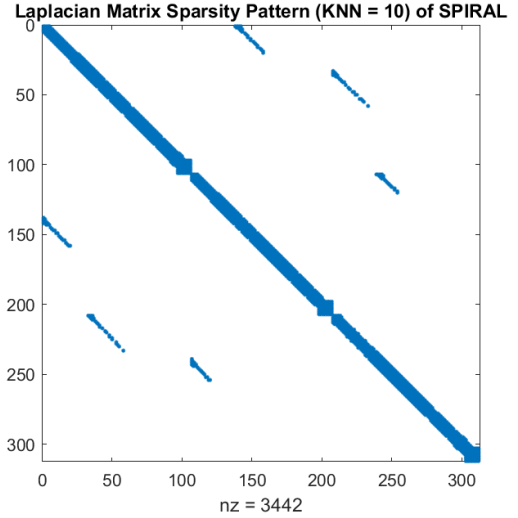


Fig. 3. Sparsity patterns of the Laplacian matrix for the spiral dataset with KNN = 10 (top) and KNN = 40 (bottom). The increase in KNN results in a denser Laplacian matrix, as shown by the higher number of non-zero elements (nz).

$\text{complete, average}\}^4$, selecting the best configuration based on the average silhouette score obtained from each - the desired number of cluster to extract from the aggregation tree was also set to M .

No significant differences in the results were found between the two methods, as both achieved the same silhouette scores across all datasets. This is likely due to the simple structure of the datasets, combined with the correct tuning of k and M , which effectively respected their inherent structure. The final silhouette scores are reported in Table I.

V. CONCLUSIONS

We presented an implementation of spectral clustering; from constructing similarity matrices to computing their eigenvalues and applying clustering algorithms in the corresponding reduced eigenspace. To extract eigenvalues, we implemented

⁴Official MATLAB documentation for hierarchical clustering parameters.

TABLE I

COMPARISON OF SILHOUETTE SCORES AND CALCULATION TIMES OBTAINED USING K-MEANS AND HIERARCHICAL CLUSTERING WITH WARD'S METHOD FOR THE THREE DATASETS. IN ALL DATASETS, THE SILHOUETTE SCORES ARE THE SAME, UNDERLYING THAT BOTH K-MEANS AND HIERARCHICAL CLUSTERING CONVERGE TO THE SAME EXACT CLUSTERING SOLUTION.

Silhouette Score	<i>k-means</i>	<i>Hierarchical</i>
Spiral ($M=3$)	0.9567	0.9567
Circles ($M=3$)	0.9842	0.9842
Torus ($M=2$)	0.9934	0.9934

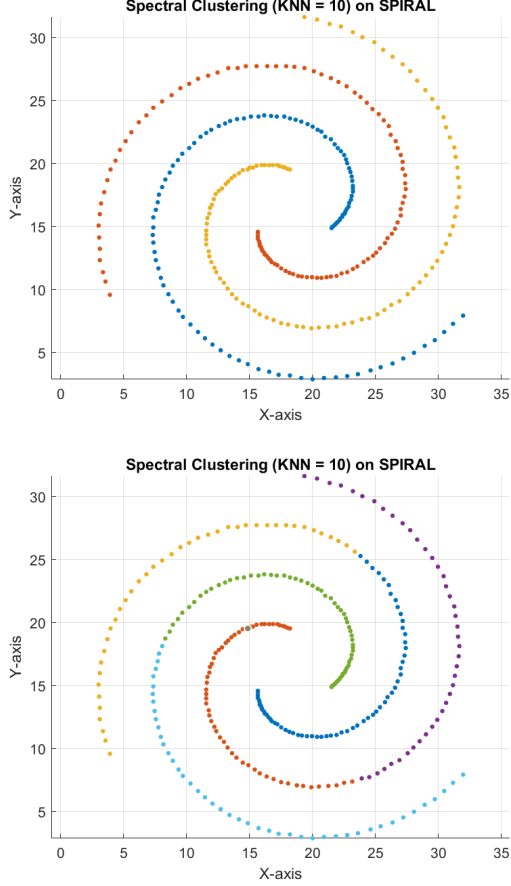


Fig. 4. Spectral clustering on Spiral dataset for different values of M : $M = 3$ (top), $M = 6$ (bottom). The algorithm successfully divides each arm of the spiral into separate clusters due to the consistent density and connectivity along the spiral arms. This ensures that the eigenvectors of the Laplacian matrix effectively capture the intrinsic structure, allowing for an even partitioning of the dataset.

deflation method, we furthermore presented its pseudocode, which relies on inverse power method to extract the eigenvalues of each subsequent sub-matrix extracted during its execution. The methodology was validated on three synthetic datasets where traditional methods fail, demonstrating its ability to cluster data points effectively. We experimented to various values for the hyperparameters encountered throughout the process, fine-tuning them to reach the highest average silhouette score possible. Overall, we achieved high silhouette scores, with values close to 1 for all the results provided.

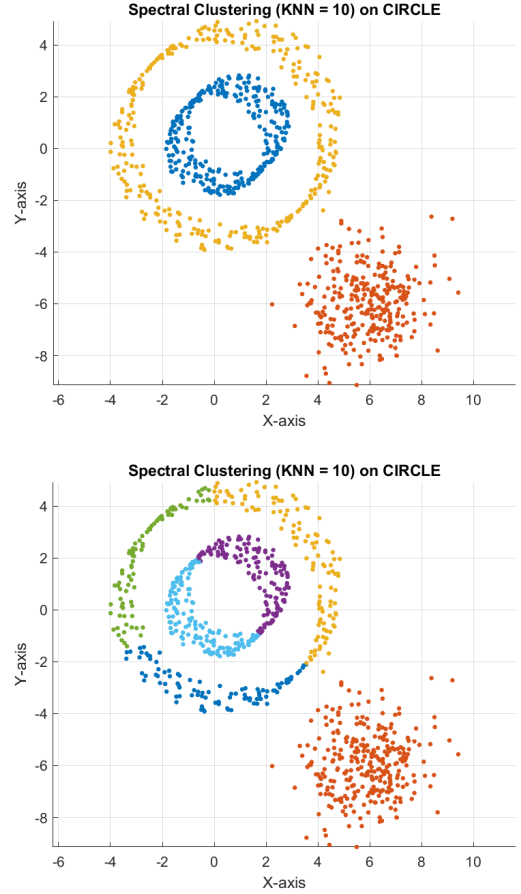


Fig. 5. Spectral clustering on Circles dataset for different values of M : $M = 3$ (top), $M = 6$ (bottom). Only the circles situated in the top left corner of the scatter have been divided in two distinct clusters, while the noise cloud residing in the bottom right corner has been considered as a unique cluster. Evidently the corresponding eigenvectors of the Laplacian treat this region as a single connected group, but do not exhibit enough variation to subdivide it due to low values of the connections.

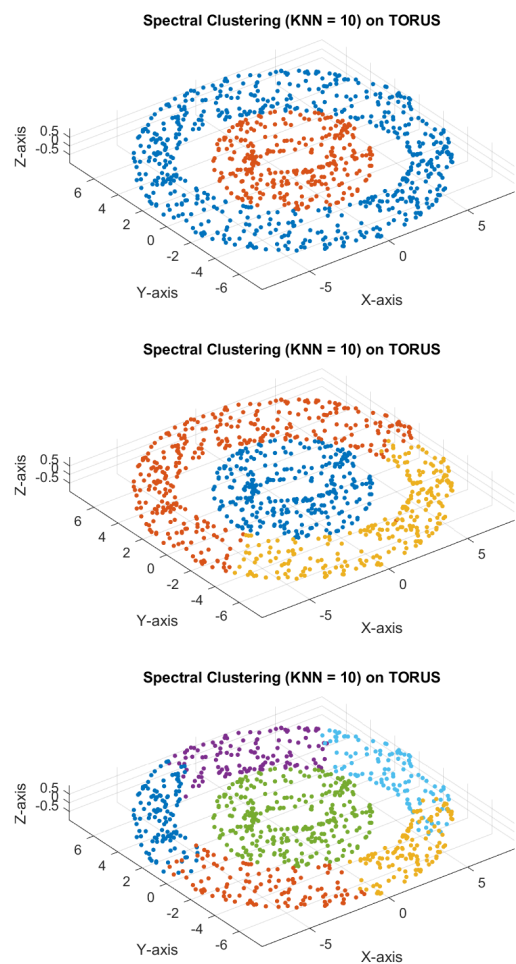


Fig. 6. Spectral clustering on Torus for different values of M : $M = 2$ (top), $M = 6$ (bottom). The clustering of the former plot separates the inner and outer toroidal layers. Latter subdivisions focus on the outer toroidal layer, as it has a greater density and more pronounced local variations in connectivity, making it more prone to further splits compared to the uniform inner layer.

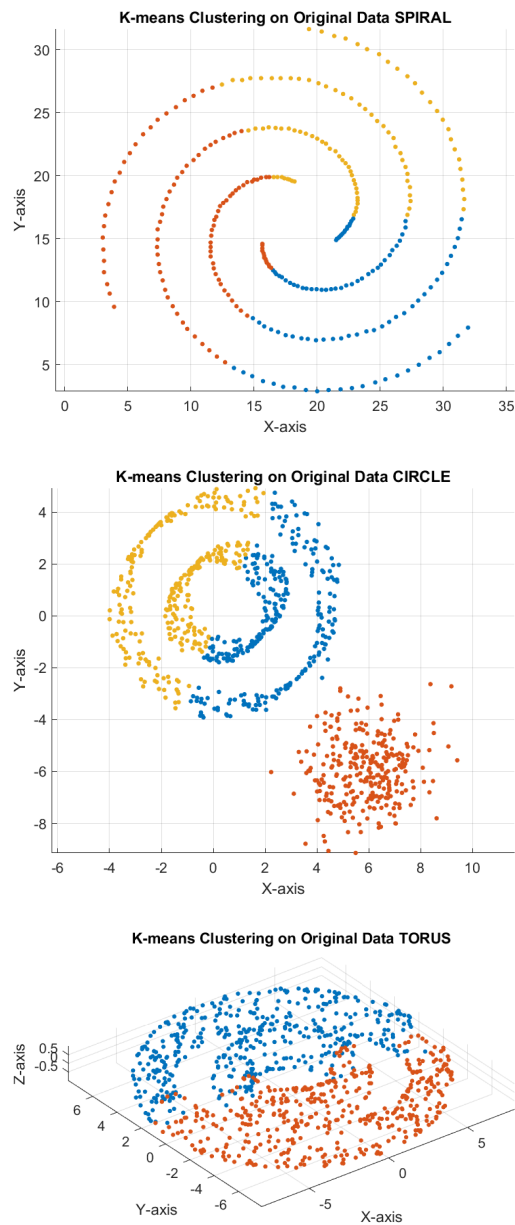


Fig. 7. K-means clustering on the same datasets using $k = 3, 3, 2$. We can see how these solutions perform poorly compared to Spectral Clustering; ignoring the underlying structure of our datasets.